SafeRouter ReplayLab

Hack Apertus 2026 | Track 2B | Sumplus - Jakob

SafeRouter ReplayLab converts explicit retry/cancel path descriptions into bounded test candidates with a locally served Apertus 1.5 model. A predefined protocol compares a separate reference and six deliberately faulty synthetic routers. Confirmed counterexamples can be minimized, downloaded and replayed without a model or network. This is a new local prototype started on 15 October 2026; no real provider calls, customer records or production billing are used. The comparison does not establish superiority over Hypothesis.

1 / A bounded, inspectable pipeline

Requested path -> local Apertus proposal -> schema and resource validation -> separate reference/target execution -> minimized divergence -> independent replay bundle. The model does not decide correctness or execute code.

On-premise is the tested target architecture. Deterministic replay works with Docker network disabled. Live local inference requires separately provisioned model assets. A fully air-gapped live deployment and Swiss cloud are not claimed.

The application exposes actual recorded responses, original timings, target traces and downloadable replay packages. Manual edits are labelled manual; model failure is explicit. A negative result is limited to the current sequence.

Protocol v1

request creates a logical request. Unique keys are independent. Duplicate request returns duplicate without effects. Uninterrupted request completes and charges exactly one unit. A break before headers or after first chunk leaves interrupted, charge zero, no pending resource. retry unknown returns unknown without effects. retry interrupted or completed completes with total charge one. retry cancelled returns cancelled without effects. cancel unknown returns unknown without effects. cancel interrupted cancels, charge zero, no pending resource. cancel completed preserves completed and its charge. Repeated cancellation is idempotent. A later retry simulates late completion after cancellation. This is synchronous local simulation, not real network concurrency or provider billing. The owner predefines this protocol. A model only translates requested paths. Reference equivalence is the oracle; it does not establish natural language specification correctness.

Protocol and synthetic defects were frozen before model inference. The detailed reference was authored with coding-assistant help; independent human validation has not been recorded.

2 / Model, data and an observed repair

Apertus 1.5 8B Q4_K_M, llama.cpp b11429 d81235049, local CPU two threads, GPU disabled, context 4096, one slot, temperature 0 seed 42. No fine-tuning. The externally supplied GGUF digest fixes the tested artifact; exact quantization origin is not independently certified.

The native adapter uses apply-template, completion(return_tokens=true), and detokenize. One ordered32/33 thinking boundary is required; only the final answer tail is parsed. All candidates face independent field/type/operation/resource validation.

Model input includes the DSL, one non-streaming request/cancel example and the requested path. Target labels/source, gold paths, observations and decisions are never provided.

Twelve newly authored tasks include ten executable paths, two rejection cases, and German text. All data is synthetic. Six complete injected fault families and two clean labels were frozen before output; original commit976c383.

Initial adapter: disabled thinking produced no required boundaries, so all 12 answers were rejected. A null-comparison scoring bug is explicitly corrected by a separate rescore, retaining original raw rows. The repaired adapter enables thinking and clarifies op values with one teaching example. Same-task repetitions are disclosed, not treated as untouched heldout tests.

Repaired set: 4/12 executable candidates; 3/12 exact task outcomes including expected rejections. Repaired inference 1781.52s; failed initial cohort plus diagnostic 328.54s; model startup 10.49s. All 25 attempts count in comparison-pipeline cost 2120.54s.

Weights were pre-existing, so download is excluded. Specification authoring and preparation were not timed. No provider fees, production-provider API usage, customer data or productivity claim.

3 / Complete bounded comparison

A: Hypothesis 6.151.9 stateful generation. B: the same generator after model seeds. Both arms share DSL, reference, targets, oracle and deletion shrinker. Native Hypothesis failure shrinking is disabled for shared accounting. This is a real but bounded stateful configuration, not exhaustive tool tuning.

Five fixed generator seeds 17,29,43,71,101; 12 steps per example; 800 target operations or 10 seconds per target/seed/arm. All candidate/shrinking operations count. Full startup and failed/repaired generation costs are divided equally across all eight targets in each replicate. The same model set is reused across generator seeds. The time view applies cutoffs to measured generation latency, not five new model deadline launches.

Budget	Arm	Fault trials found	Clean false positives
calls	A	30/30	0
calls	B	30/30	0
time	A	30/30	0
time	B	0/30	0

Injected family	Calls A	Calls B	Time A	Time B
retry_double_charge	5/5	5/5	5/5	0/5
late_completion	5/5	5/5	5/5	0/5
cross_key	5/5	5/5	5/5	0/5
unknown_cancel	5/5	5/5	5/5	0/5
stream_residue	5/5	5/5	5/5	0/5
reject_legal	5/5	5/5	5/5	0/5

Median time to first counterexample: call view A 0.199s, B 265.068s, including model cost. Time view A 0.178s; B has no observation because its budget expires before any target call. The zero B-time false-positive count is not a tested clean-case accuracy rate. Extra discovered families in B:0. Total target operations in the call view: A 11690, B 9014; these are local synthetic operations, not paid provider calls.

All 160 expected rows are present. Independently replayed detected traces: 90. False positives on both clean labels:0. Full raw per-seed results and undiscovered targets are included. No statistical superiority is established.

4 / Replaying a real generated candidate

This example comes from an actual accepted Apertus output. It targets a deliberately injected synthetic fault. Full original candidate, final answer, timing and complete observations are retained in results/demo-counterexample.json and model-candidates.jsonl.

Target: late_completion

```
{
  "ops": [
    {
      "op": "request",
      "key": "request1",
      "stream": true,
      "break_at": "after_first_chunk"
    },
    {
      "op": "cancel",
      "key": "request1"
    },
    {
      "op": "retry",
      "key": "request1"
    }
  ]
}
```

Independent command: python track_2b/src/replay.py bundle.json

The bundle includes original and minimized operations, per-step reference/target observations, provenance and SHA256. A fresh Python process validates it without a model. SHA256 detects accidental edits; it is not an authenticated signature.

Controls and deployment evidence

Six positive canaries each detect a known injected fault; correct behavior is clean. A deliberately disabled oracle fails those canaries. Tampered bundles, forbidden operations, invalid field types, too many keys and excessive sequence lengths are rejected.

Docker image builds locally. Root make run starts the service. A network-none Docker run replays recorded evidence and controls. The local adapter rejects non-local hosts. Three runtime socket snapshots show loopback communication only; they are observations, not an exhaustive packet or syscall audit.

A separate post-evaluation live UI functional check generated a valid one-request sequence in 27.37s. The correct target stayed clean, the injected legal-rejection target diverged, and the downloaded bundle replayed in a fresh Python process. This 26th model attempt and its time are separate from the 25-attempt comparison pipeline, not another independent performance trial.

Reference equivalence does not establish the real-world billing contract. The synchronous simulation does not test genuine concurrent transport, asynchronous callbacks or real streaming.

5 / Reproducibility, limits and sources

Code version: fb782abe4bd18cb996a9de816aff6abc125b4ddc

Tested host: Apple M1 Max, 32GiB RAM, macOS 26.1; native arm64 runtime and Python3.12.14. Shared system load was not controlled.

From a clean checkout, run make run at repository root. The Docker application listens on localhost19564. Recorded-response inspection/replay works without a model. Live generation needs separately provisioned local Apertus and LLM_NAME, LLM_BASE_URL, LLM_API_KEY settings. See docs/REPRODUCE.md for exact commands and model digest.

This prototype tests a narrow, synthetic protocol. Schema acceptance is distinct from path fidelity. Repaired repetitions are not independent heldout results. The reference itself needs independent owner review. No production fault, customer saving, real-provider effect, security audit or Swiss cloud deployment is claimed.

Next work: independently review the protocol; add real asynchronous transport; freeze new untouched tasks; compare stronger baseline settings and measure repeated model draws before estimating incremental practical value.

Authorship and licensing

New code/tasks were created within this project on 15 October 2026. Official template copyright is retained. Code/template: Apache-2.0. Report/documentation: CC-BY-4.0. Authored synthetic task data: CDLA-Permissive-2.0. External model/runtime/dependency licences apply.

Codex assisted implementation, protocol, experiment plumbing and local materials. Apertus 1.5 alone produced runtime model candidates and did not grade them. No old competition project was reused.

References

https://github.com/HackApertus/project-template

https://hackapertus.ch/online-hack

https://huggingface.co/swiss-ai/Apertus-v1.5-8B

https://hypothesis.readthedocs.io/en/latest/stateful.html

https://github.com/ggml-org/llama.cpp/tree/master/tools/server